Road Condition AI: PRD

Oct 25, 2026 · @Siddharth

Overview

Road Condition AI is a student-scale machine learning project that reads road condition from photos, then measures how well a road damage detector survives a move to another country.

Research question: a detector trained on Japanese road photos loses accuracy on Norwegian ones. How much, and what is the cheapest way to get it back?

The project has three parts:

1. **Surface classifier.** Predict dry, wet, water, fresh snow, melted snow or ice from a cropped road photo, comparing a classical baseline, a small CNN and a fine-tuned pretrained model.
2. **Cross-country damage detector.** Train a YOLO detector on one country, test it on others, and compare three fixes: stronger augmentation, a few labelled target images, and pseudo-labelling.
3. **Demo.** A small web app where a visitor uploads a road photo and sees the predicted condition, a confidence value and a heatmap of where the model looked.

The point is a portfolio piece that shows the full ML workflow on a problem close to Univrses' road-condition products. It is not a production system.

Problem and users

Road authorities need to know where roads are icy, wet or damaged, and manual inspection is slow and expensive. Cameras plus ML can do it continuously, but models trained in one country often degrade in another, and labelling new data for every country is costly.

User	What they need	How the project serves them
Recruiter or mentor	Evidence you can do the work, checked in about two minutes	Live demo, clean README, honest results table
You, in an interview	Something you can explain end to end	Every design choice is tracked in an experiment log

User	What they need	How the project serves them
Road authority (the imagined end user)	Current road condition, and trust in the model	Condition label with calibrated confidence and a heatmap

Link to Univrses. Univrses' 3DAI product builds live maps of road condition for road transport authorities. The thesis topics for spring 2027 include radar-based road surface condition estimation and cross-dataset matching. This project covers the camera version of the first and the domain-shift question behind the second. It does not claim to be either.

Goals, non-goals and scope

The project succeeds if every claim in the README is backed by an experiment you ran and can explain.

Goals

- Build and compare three classifiers on a balanced RSCD subset, and report per-class results, not only accuracy.
- Quantify the cross-country accuracy drop of a YOLO detector, and compare three recovery methods at several label budgets.
- Ship a public demo and a reproducible repository (one command to set up, one to train).

Non-goals

- Radar processing. Public radar data for road surface condition is hard to find, so radar appears only as future work.
- Real-time or on-vehicle deployment.
- Beating published state of the art. A fair, well-explained comparison matters more than a leaderboard score.
- Visual localization. It is an optional stretch notebook, not part of the core scope.

Scope guardrails

- Use a subset of 30,000 to 50,000 RSCD images, not the full set, so each run fits in a free Colab or Kaggle session.
- Keep the friction label (dry, wet, water, snow, ice) as the main task. Material and unevenness are optional extensions.
- Cap the cross-country study at three target countries.

Data

Two public datasets cover the project: RSCD for classification and RDD2022 for cross-country detection.

	RSCD	RDD2022
Task	Image classification	Object detection (bounding boxes)
Size	About 1 million patches, 240 x 360 px	47,420 images, 55,000+ damage instances
Source	Vehicle camera in China, cropped to road surface (IEEE DataPort)	Japan, India, Czech Republic, Norway, USA, China (arXiv)
Labels	27 classes from friction (dry, wet, water, fresh snow, melted snow, ice), material and unevenness	Longitudinal crack, transverse crack, alligator crack, pothole
Official split	About 960k train, 20k validation, 50k test	38,385 train, 9,035 test (Dataset Ninja)
Used here	30k to 50k balanced training subset, friction label	Japan as source; Norway, Czech Republic, USA as targets

Per-country image counts in RDD2022 are Japan 13,133, Norway 10,201, India 9,665, USA 6,005, Czech Republic 3,538, plus China motorbike 2,477 and China drone 2,401.

Data rules

- Keep the official RSCD validation and test sets untouched. Subsample only the training set.
- Check for near-duplicate frames across splits. Consecutive frames from one drive can leak between train and test; confirm how the official split was built before trusting it.
- Confirm whether RDD2022 test labels are public. If they are not, build your own held-out split from the labelled images, per country.
- Fix random seeds and save the exact file lists for each subset, so every number can be reproduced.
- Check each dataset's licence and citation requirements before publishing the repository or demo images.

Functional requirements

Each requirement is testable: it either exists in the repository and runs, or it does not. Priority: **P0** is required, **P1** is strongly wanted, **P2** is a stretch.

ID	Requirement	Priority
C1	EDA notebook: class counts, sample grid per class, image statistics, split sanity checks	P0
C2	Classical baseline: colour histogram and HOG features into logistic regression and random forest	P0
C3	Small CNN written in PyTorch with your own training loop, loss curves and early stopping	P0
C4	Fine-tuned pretrained model (ResNet-50 or EfficientNet) with the same evaluation as C2 and C3	P0
C5	Evaluation report: macro-F1, per-class precision and recall, confusion matrix	P0
C6	Imbalance handling compared: none, class-weighted loss, oversampling	P1
C7	Hyperparameter sweep over learning rate, batch size and augmentation strength, logged to an experiment tracker	P1
C8	Grad-CAM heatmaps for correct and wrong predictions	P1
C9	Calibration: reliability diagram and expected calibration error, before and after temperature scaling	P1
C10	Optional ViT or DINOv2 backbone comparison	P2
D1	YOLO detector trained on the source country, with mAP reported per target country	P0
D2	Recovery experiment: stronger augmentation	P0
D3	Recovery experiment: fine-tuning with 10, 50 and 200 labelled target images, repeated over at least 3 random seeds	P0
D4	Recovery experiment: pseudo-labelling on unlabelled target images with a confidence threshold	P1
D5	Learning curve: target accuracy against number of labelled target images	P0

ID	Requirement	Priority
W1	Demo: upload an image or pick a sample, get condition, confidence and heatmap	P1
W2	Demo shows its limits: a visible note on training data, country and unseen conditions	P1
W3	Model exported to ONNX and served from a lightweight API	P2
R1	README with setup, results tables, plots, limitations and future work	P0
R2	2 to 4 page write-up of the cross-country study	P0
L1	Optional stretch: SIFT with RANSAC versus a learned matcher on images of one place under different conditions	P2

Experiment plan and expected results

The expected results are hypotheses to test, not numbers to report. No score is promised in advance, because it depends on your subset, training time and settings. Whatever you measure is what you publish, including disappointing results.

#	Hypothesis	Deliverable	Counts as confirmed if
H1	Fine-tuned pretrained model beats the small CNN, which beats the classical baseline	Table of macro-F1 and accuracy for C2, C3, C4	Order holds on the untouched test set over 3 seeds
H2	Visually similar classes are confused most (for example wet with water, fresh snow with melted snow)	Confusion matrix and a short written reading of it	The top confusions are between neighbouring friction levels
H3	Class imbalance hurts the rare classes, and weighting or oversampling helps them	Per-class F1 with and without correction	Rare-class recall rises without a large macro-F1 loss

#	Hypothesis	Deliverable	Counts as confirmed if
H4	Raw model confidence is over-confident, and temperature scaling improves calibration	Reliability diagram and expected calibration error	Calibration error drops after scaling on held-out data
H5	Some Grad-CAM maps show the model using shortcuts such as sky, shadows or vehicle parts	Gallery of right and wrong cases with maps	At least a few wrong predictions are explained by where the model looked
H6	A detector trained on Japan loses accuracy on Norway, the Czech Republic and the USA	mAP per target country next to source mAP	Target mAP is lower than source mAP, with the gap stated per country
H7	A small number of labelled target images recovers a large part of the gap	Learning curve at 0, 10, 50, 200 labels, mean and spread over seeds	The curve rises, and you can say where it flattens
H8	Fine-tuning with labels beats augmentation alone, and pseudo-labelling lands in between	Comparison of D2, D3, D4 at equal budget	The ranking holds across seeds, or you report that it does not

A reference point for H6: the winning entry of the 2022 road damage challenge reached an F1 of 76.9% across all six RDD2022 countries (Geoscience Data Journal). A student-scale model trained on one country should not be expected to match it, and does not need to.

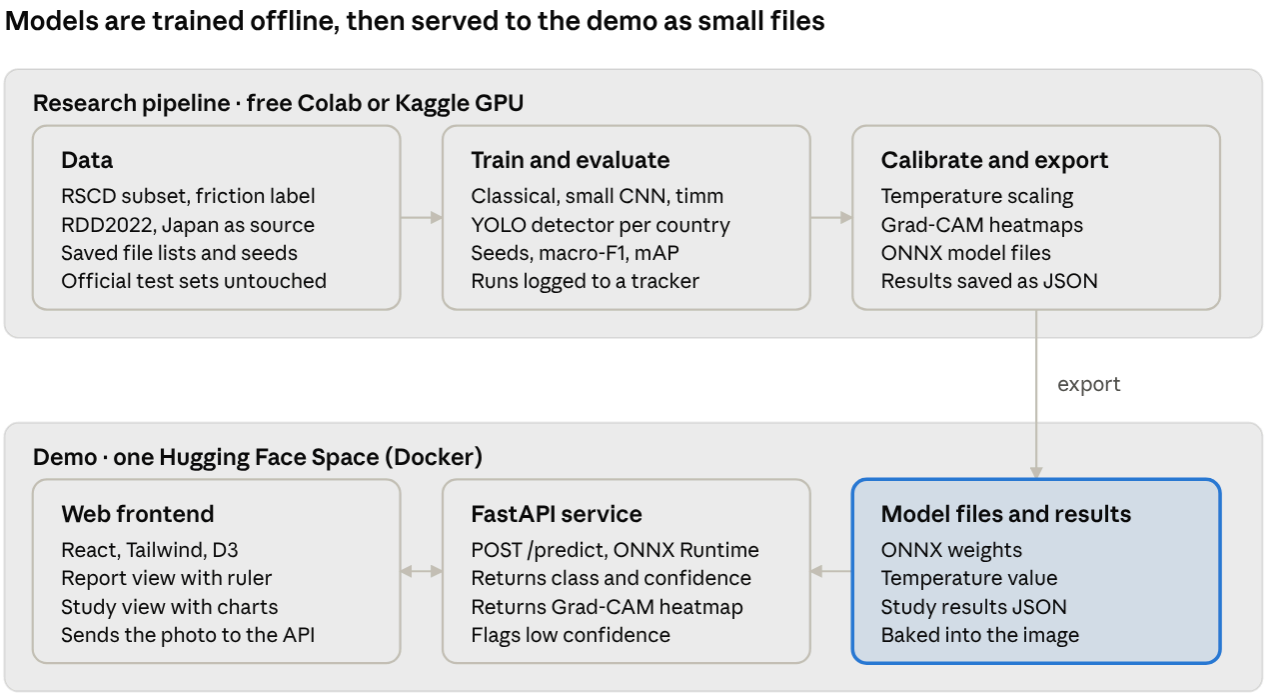
Metrics

- Classification: macro-F1 as the headline, then accuracy, per-class precision and recall, expected calibration error.
- Detection: mAP at IoU 0.5 and at 0.5 to 0.95, plus F1 at a fixed confidence threshold.
- Domain shift: relative gap, defined as (source mAP minus target mAP) divided by source mAP, and share of gap recovered by each method.

Reporting rules

- Report mean and spread over at least 3 seeds, not a single best run.
- Tune on validation data only, and touch the test set once per final model.
- State clearly when a result is not significant or a hypothesis fails. A refuted hypothesis with a good explanation is a valid finding.

System architecture



system architecture · 2 zones, 6 components

Research and serving meet only at the highlighted box: the exported model files and results JSON. The demo shows only what those files contain, so retraining never touches the web code.

Tech stack

Python and PyTorch carry the research, and a small FastAPI service plus a hand-built web frontend carry the demo. Everything below is free to use for a student project, and training runs on a free Colab or Kaggle GPU.

Layer	Choice	Why	Alternative
Language	Python 3.11	Standard for ML, every library below supports it	None needed
Deep learning	PyTorch and torchvision	Most used in research, and what a thesis supervisor expects	JAX (steeper curve)

Layer	Choice	Why	Alternative
Pretrained models	timm	One API for ResNet, EfficientNet, ViT and DINOv2 backbones	torchvision.models
Classical ML	scikit-learn, scikit-image (HOG)	Clean baseline with almost no code	OpenCV for features
Augmentation	Albumentations	Fast, supports boxes for detection, easy weather-style effects	torchvision.transforms v2
Detection	Ultralytics YOLO	Quick training, built-in mAP, easy fine-tuning. Licensed AGPL-3.0, so check the terms before public release	torchvision Faster R-CNN
Explainability	pytorch-grad-cam	Ready Grad-CAM for CNNs and ViTs	Captum
Calibration	Temperature scaling written by hand (about 20 lines)	Teaches the idea and needs no extra dependency	netcal
Experiment tracking	Weights & Biases or TensorBoard	Compare runs, seeds and sweeps in one place	MLflow
Config and data	YAML configs, saved file lists, Git LFS or DVC for small artefacts	Reproducible subsets	Plain scripts
Compute	Google Colab or Kaggle notebooks	Free GPU, no local hardware needed	A university GPU server
Model export	ONNX and ONNX Runtime	Small, fast CPU inference for the demo	TorchScript

Layer	Choice	Why	Alternative
API	FastAPI	Simple, typed, one endpoint is enough	Flask
Frontend	Vite, React and Tailwind CSS, with D3 for the study charts	React components suit the photo, ruler and toggle. Tailwind is set to the survey-sheet colours and fonts, so the page avoids the stock look. D3 does the chart maths and React draws the SVG	Svelte (smaller bundle, but fewer people know it)
Hosting	One Hugging Face Space (Docker) serving the API and the built frontend	Free, one deployment, public link for the CV	Render or Fly.io
Quality	Git and GitHub, ruff, pytest for data and metric code, GitHub Actions	Shows engineering habits cheaply	None needed

Build order. Notebooks and scripts first, since research is the core. Move shared code into a small `roadcond` package once two notebooks need it. Build the frontend last, against a model that already works.

UI and visual design

The demo is designed as a road survey sheet: one photo treated like a specimen, one friction ruler that reads from dry to ice, and a footer that states the model's limits. It avoids the usual AI-demo look of purple gradients, glass cards, stock icons and a centred upload box.

Mockups of the report, empty, study and phone screens are in the Road Condition Survey Sheet design. Values on them are placeholders until the model runs.

Concept. A pavement inspector fills in a survey sheet: a photo, a measurement, a stamp. The page does the same. The photo sits left with crop marks and registration ticks. A vertical friction ruler sits right, with a tick for each class and a marker at the predicted one. A rubber-stamp style label gives the verdict.

Screens

1. **Report.** Empty state with three sample photos and an upload drop area drawn as a blank specimen frame. Result state with the photo, heatmap toggle, ruler, verdict stamp and confidence.
2. **Study.** The cross-country result: a slope chart from source mAP to target mAP per country, and a learning curve over 0, 10, 50 and 200 labels. Built from your own measured numbers only.
3. **Limits.** A footer strip and an expandable panel: training data, country, what the model has not seen, and the calibration note.

States to design: empty, loading, result, low confidence (shows "Not sure" with the top two classes, never a forced label), not a road photo, upload error, and offline.

Typography

Role	Family	Weight	Use
Display and class labels	Big Shoulders Display	700 to 800, uppercase	Page title, verdict stamp, large numbers. Condensed and industrial, close to road-sign lettering
Data and captions	IBM Plex Mono	400 and 500	Probabilities, file info, ruler ticks, model details
Reading text	Public Sans	400 and 600	Explanations and the limits panel

All three are open-licence Google Fonts. Display sizes run from 56 to 96 px with tight line height of about 0.9. Body is 16 px at 1.5 line height. Mono data is 13 px with 4% letter spacing, and nothing goes below 11 px.

Colour

Role	Value	Notes
Paper	#EDE8DC	Page background, warm like survey paper
Asphalt ink	#16181A	All text and rules. About 14.5:1 contrast on paper
Lane-marking yellow	#F2B705	The only accent: the marker, focus rings, the primary button. Always a fill with ink text on it, never yellow text on paper (about 1.5:1)

Role	Value	Notes
Condition tints	Dry warm grey, wet steel blue, water deep blue, fresh snow white with ink outline, melted snow pale slate, ice light cyan	Fills for ruler segments. Every class also has a distinct pattern and a text label, so colour is never the only signal

A dark variant uses asphalt as the background and paper as the text. Verify each text and background pair with a contrast checker before release.

Motion. The ruler marker travels to the predicted class in about 300 ms with an ease-out curve. The heatmap crossfades in 180 ms. The stamp lands with a short 120 ms scale settle. All motion switches off under the reduced-motion setting.

Accessibility. Full keyboard use, visible yellow focus ring on ink, alt text that states the predicted class and confidence, and results announced to screen readers through a live region.

Honesty in the interface. Show the confidence as a number with its meaning ("calibrated on held-out data"), and show the limits line on every result. The demo must not look more certain than the model is.

Timeline, risks and success criteria

The plan is about five weeks part-time, and you can apply after week 2. These durations are estimates and shift with your prior experience.

1. **Week 1.** Set up the repository and tracking. Do C1 to C3: EDA, classical baseline, small CNN.
2. **Week 2.** Do C4 to C9: pretrained model, evaluation, imbalance, sweep, Grad-CAM, calibration. **Gate:** Part 1 is solid, so add the repository to your CV and start applying.
3. **Weeks 3 and 4.** Do D1 to D5: source detector, per-country mAP, three recovery experiments, learning curve over multiple seeds.
4. **Week 5.** Build the demo (W1, W2), write the README and the 2 to 4 page study (R1, R2), record a 60-second screen capture.

Deliverables: the GitHub repository, the write-up, the live demo link, and one results figure for the CV.

Risks

Risk	Mitigation
Train and test frames leak, inflating scores	Check near-duplicates across splits, keep the official test set untouched
Free GPU sessions time out	Use the subset, save checkpoints each epoch, use mixed precision
RDD2022 test labels are not public	Build your own per-country held-out split from labelled images
Small label budgets give noisy curves	Repeat over at least 3 seeds and report spread
Scope creep into radar or localization	Keep both as P2 and future work
Licence terms (Ultralytics AGPL-3.0, dataset terms)	Read them before publishing code, weights or sample images
Demo overclaims, and RSCD comes from China only	Show the limits line, add the low-confidence state, and test on a few Nordic road photos you are allowed to use, reporting failures openly

Success criteria

- All P0 requirements exist, run, and are described in the README.
- A clean clone reproduces the main results table with the documented commands.
- Results carry mean and spread over at least 3 seeds.
- The demo is public and shows its limits.
- You can explain each design choice and each result in a few minutes, without notes.

Presenting it honestly. Keep real commit dates, say in your cover letter that the project was built in response to the posting, and describe it as camera-based surface classification plus a domain-shift study, not as radar work.

Sources

Dataset facts come from search-result excerpts of these pages, checked 2026-10-25. Read the original pages and licences before you publish anything.

- RSCD on IEEE DataPort: size, image format, 27 classes, split sizes
- RoadMamba paper: friction, material and unevenness label definitions
- RDD2022 paper on arXiv: image count, countries, damage classes
- RDD2022 on Dataset Ninja: per-country counts and train and test split sizes

- RDD2022 in Geoscience Data Journal: challenge winner F1 of 76.9%

The tool choices, time estimates and design values in this document are my recommendations, not sourced facts.